

KIẾN TRÚC HƯỚNG DỊCH VỤ

Giới thiệu & Sơ bộ về SOA

TS. Lê Khánh Trình

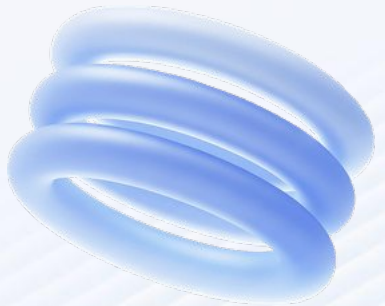
NỘI DUNG

01. Giới thiệu môn học

02. Lịch sử phát triển PM

03. API và Web services

04. Các kiến trúc API phổ biến



Giới thiệu giảng viên



Doctor

LÊ KHÁNH TRÌNH

Lecturer of

Faculty of Information Technology,

University of Engineering and Technology,

Vietnam National University, Hanoi

321 - E3, Xuan Thuy Campus, UET-VNUH

trinhlk "at" vnu-mail

+33 (0) 668 772 886

Hướng nghiên cứu

- **UI Testing**:
  - Tự động hóa quy trình kiểm thử cho các ứng dụng Web
- **Security Testing**
- **Smart Contract Quality Assurance:**
  - Xác định dấu hiệu lừa đảo, kiểm chứng chất lượng SC
- **Software Verification:**
  - Sử dụng các Model Checker, Theorem Prover để xác định tính đúng đắn, tính nhất quán của bộ đặc tả hoặc các chức năng triển khai

Nội dung môn học

NỘI DUNG CHÍNH	MỤC TIÊU SINH VIÊN ĐẠT ĐƯỢC
Giới thiệu API, Web Services	Hiểu vai trò API trong hệ thống phần mềm
REST & HTTP Fundamentals	Thiết kế request/response đúng chuẩn
Nguyên tắc thiết kế API	Xây dựng API nhất quán, dễ dùng
OpenAPI & Swagger	Tạo tài liệu API tự động
Data Modeling & Resource Design	Thiết kế cấu trúc dữ liệu cho API
Authentication & Authorization	Bảo mật API
Backend Implementation	Xây dựng backend từ spec
API Testing	Kiểm thử tự động, hiệu năng
API Versioning	Quản lý thay đổi API
Service Operation	Deploy, monitoring, bảo mật production
API Design Patterns	Áp dụng mẫu thiết kế cho các tình huống thực tế
API as a Product	Xem API là sản phẩm kinh doanh
Dự án nhóm	Triển khai API hoàn chỉnh + quản lý vòng đời

Cách học

Chia nhóm (tối đa 5 sinh viên/nhóm): **23020168 Trần Thị Thanh Vân**

- Lớp trưởng tạo file chia nhóm

Lịch trình:

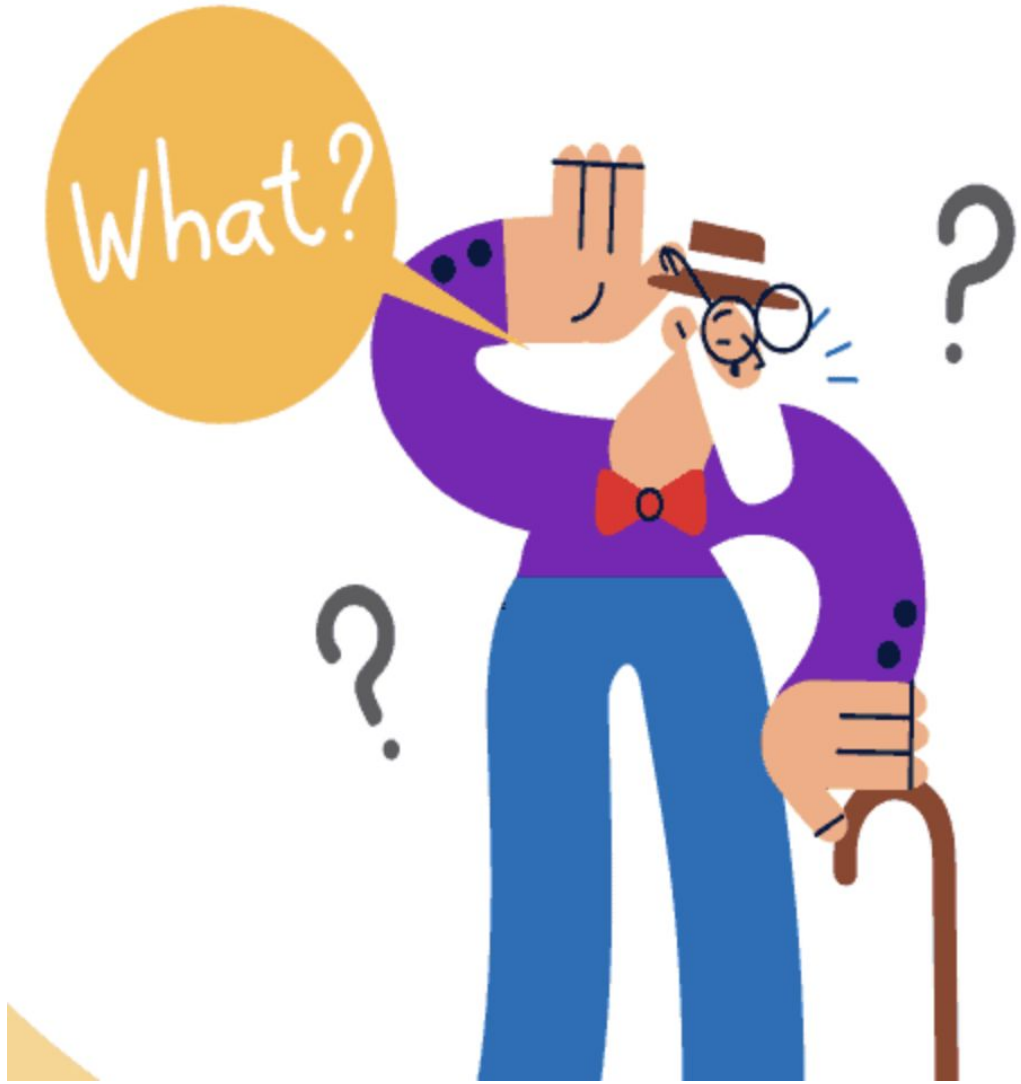
- Chuẩn bị cho nội dung buổi học theo các ý cho trước (tự làm slide)
  - Thầy gọi ngẫu nhiên các bạn lên trình bày và đặt câu hỏi
  - Chủ động, tích cực sẽ được cộng điểm. Ngược lại, bị trừ điểm
  - **Yêu cầu slide nhóm:**
    - Dựa trên đề cương, học liệu
    - Có ví dụ thực tế, hình ảnh minh họa do sinh viên **tự code** và cho vào
- Tuần 13 → 15: Các nhóm trình bày dự án

Tính điểm

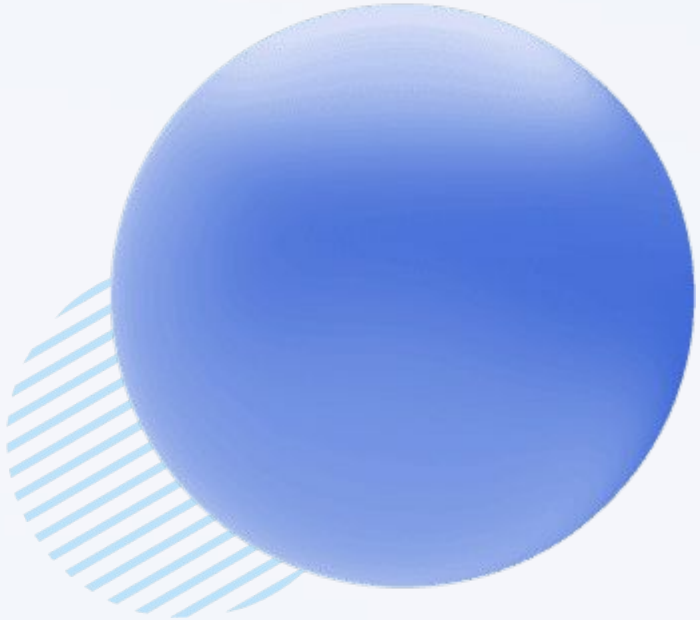
- Điểm thường xuyên + kiểm tra: 30%
  - Điểm danh đủ thì tối đa 7đ
  - Điểm cộng:
    - Xung phong trả lời câu hỏi
    - Đặt câu hỏi cho nhóm bạn
    - Cướp điểm từ việc bạn khác không trả lời được
  - Điểm trừ:
    - Không chuẩn bị bài
    - Không trả lời được câu hỏi
  - Cấm thi: gian lận, tiếp tay cho gian lận
- Điểm slide + trình bày: 30%
  - Slide nhóm nhưng code demo là cá nhân
  - Tạo Github public repo tên: **2526II_INT3505_1**
- Điểm bài tập nhóm: 40%

Tài liệu

- Học liệu tham khảo:
  - JJ Geewax, *API Design Patterns*
  - Bruno Pedro, *Building an API Product*
  - James Higginbotham, *Principles of Web API Design*
- Công cụ sử dụng:
  - Swagger / OpenAPI
  - Postman / Newman
  - Node.js, Mongoose (hoặc tương đương)
  - Git, GitHub (quản lý dự án)
- Hệ thống hỗ trợ:
  - LMS (Diễn đàn thảo luận, nộp bài, tài liệu)


What?

Kiến trúc phần mềm



Từ tập trung đến phân tán

Monolithic Architecture

Tất cả code (UI, Business Logic, Data Access) nằm chung một khối duy nhất.

Ưu điểm

Dễ bắt đầu, dễ deploy ban đầu

Nhược điểm

Khó scale, một lỗi nhỏ có thể làm sập toàn hệ thống, khó làm việc nhóm lớn

Microservices Architecture

Chia nhỏ thành các dịch vụ độc lập, mỗi dịch vụ chịu trách nhiệm một chức năng.

Ưu điểm

Dễ scale từng phần, một lỗi không ảnh hưởng toàn hệ thống, làm việc nhóm hiệu quả

Yêu cầu

Cần API để các dịch vụ giao tiếp với nhau

Giao tiếp hệ thống phân tán

Mobile Apps

iOS, Android apps cần API để lấy dữ liệu

IoT Devices

Smart home, wearables kết nối qua API

Web Apps

Single Page Apps giao tiếp backend qua API

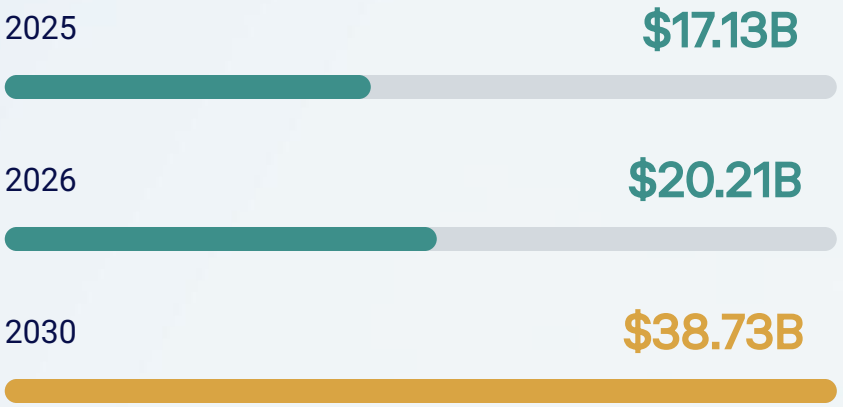
Tích hợp cross platform

B2B (Business-to-Business) và B2C (Business-to-Consumer) đều cần API để kết nối các hệ thống khác nhau.

API Economy

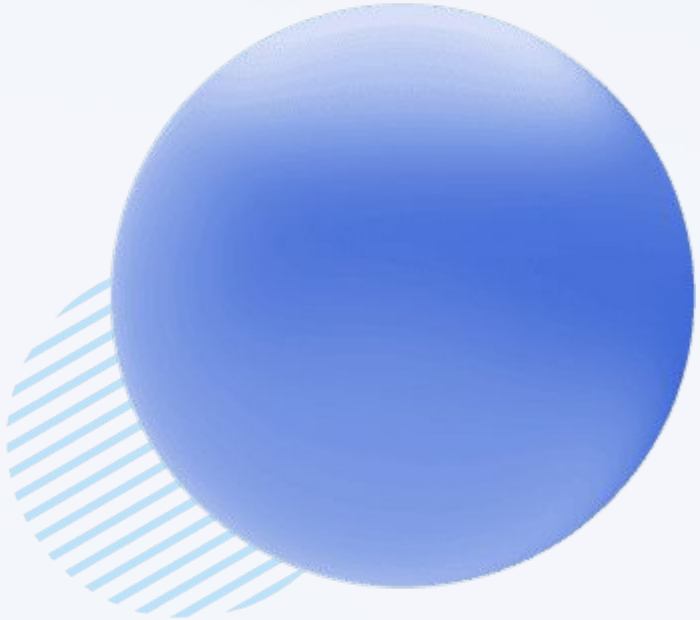
API không chỉ là kỹ thuật, mà là **sản phẩm kinh doanh**. VD: Stripe (thanh toán), Twilio (SMS), Google Maps.

API Economy Growth



CAGR: **17.9%**

API và Web services



API là gì?

API là gì?

Application Programming Interface

Giao diện lập trình ứng dụng - là tập hợp các quy tắc và cơ chế cho phép các ứng dụng giao tiếp với nhau.

Bản chất: contract

API là một **hợp đồng (Contract)** giữa người cung cấp và người sử dụng.

"Nếu bạn gửi cho tôi dữ liệu A theo đúng định dạng này, tôi sẽ trả về dữ liệu B."

Thành phần của API

Inputs (Request)

Dữ liệu đầu vào mà client gửi đến server: parameters, headers, body.

Outputs (Response)

Dữ liệu đầu ra mà server trả về: status code, body, headers.

Hành vi (Behavior)

Quy tắc xử lý: validation, business logic, error handling.

Web Services là gì?

Web Service

Dịch vụ Web

Một phần mềm cung cấp dịch vụ qua mạng **Internet/Intranet**, cho phép các ứng dụng khác nhau giao tiếp và trao đổi dữ liệu

Tiêu chuẩn mở

Hoạt động dựa trên các tiêu chuẩn web phổ biến:

- ✓ HTTP/HTTPS
- ✓ XML
- ✓ JSON
- ✓ SOAP, REST

Độc lập nền tảng

Không phụ thuộc vào:

- ✓ Hệ điều hành
- ✓ Ngôn ngữ lập trình
- ✓ Framework
- ✓ Kiến trúc hệ thống

Giao tiếp

Cho phép các ứng dụng:

- → Gửi request
- ← Nhận response
- Trao đổi dữ liệu
- Thực thi chức năng

API vs Web Services

API

Giao thức
Có thể **offline** (trong cùng một máy tính)

Ví dụ
Win32 API, thư viện Math trong C++, Java Collections API

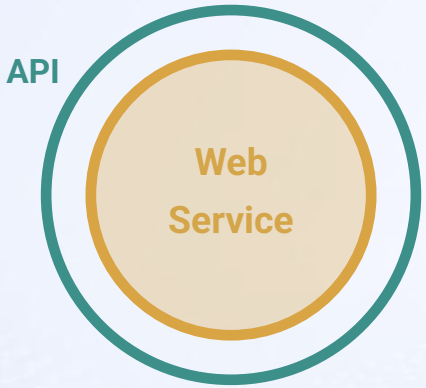
Phạm vi
Các module trong cùng một ứng dụng hoặc qua mạng

Web Service

Giao thức
Bắt buộc dùng mạng (HTTP/HTTPS)

Ví dụ
REST API, SOAP API, GraphQL Endpoint

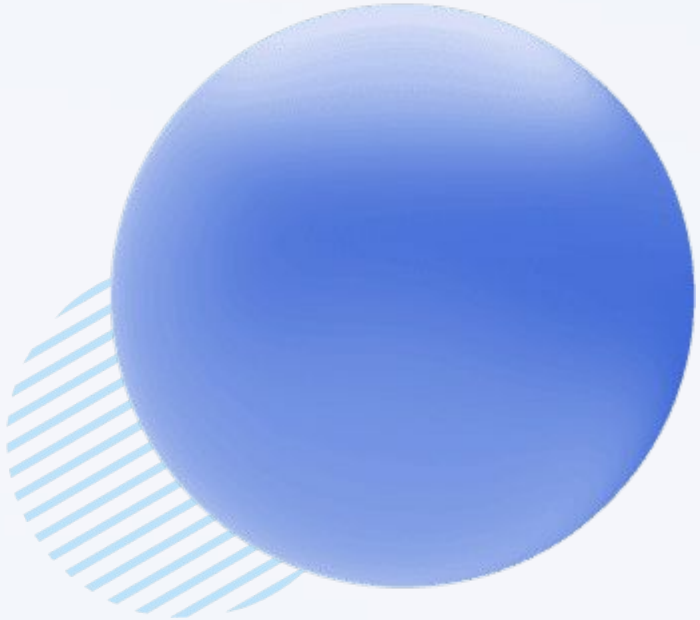
Phạm vi
Các máy tính khác nhau giao tiếp qua mạng



- Tất cả Web Services đều là API
- Không phải API nào cũng là Web Service

Các kiến trúc phổ biến

SOAP, REST, GraphQL, gRPC



SOAP (Simple Object Access Protocol)

Lịch sử

- Ra đời cuối thập niên 90
- Tiêu chuẩn hóa bởi **W3C**
- Một trong những giao thức đầu tiên

Đặc điểm

- **Giao thức** với các quy tắc nghiêm ngặt
- Dữ liệu bắt buộc là **XML**
- Định nghĩa qua file **WSDL**

Ưu điểm

Bảo mật cao (WS-Security)

Hỗ trợ mã hóa message-level, digital signatures, security tokens - phù hợp ngân hàng, y tế.

Tính toàn vẹn (ACID)

Hỗ trợ transactions: Atomicity, Consistency, Isolation, Durability - đảm bảo dữ liệu không bị mất.

Hợp đồng rõ ràng (WSDL)

Web Services Description Language định nghĩa chính xác operations, data types, endpoints.

Ví dụ về Payload SOAP

soap_request.xml

```
POST /Quotation HTTP/1.1
Host: www.xyz.org
Content-Type: application/soap+xml; charset="utf-8"
Content-Length: nnn

<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope"
               xmlns:m="http://www.xyz.org/quotations">
    <soap:Header>
        <!-- Optional: security credentials, transaction IDs, etc. can go here -->
    </soap:Header>
    <soap:Body>
        <m:GetQuotation>
            <m:QuotationsName>MiscroSoft</m:QuotationsName>
        </m:GetQuotation>
    </soap:Body>
</soap:Envelope>
```

Cấu trúc SOAP Message

1. **Envelope**
   Root element, định nghĩa namespace
2. **Header (Optional)**
   Metadata: authentication, routing
3. **Body (Required)**
   Nội dung chính: request/response

Nhược điểm:

Để gửi một thông tin nhỏ cần rất nhiều thẻ XML

Kết quả: Payload lớn, tốn băng thông, parse chậm hơn JSON.

REST (Representational State Transfer)

Lịch sử

- Giới thiệu bởi **Roy Fielding**
- Phổ biến nhất hiện nay, chiếm hơn 80% các public APIs trên thế giới

Bản chất

REST là **Architectural Style** (phong cách kiến trúc), không phải giao thức.

Tận dụng tối đa các đặc tính sẵn có của HTTP.

Đặc trưng

Dựa trên Tài nguyên (Resources)

Mọi thứ đều là **danh từ**, được định danh bằng URI:

`/users` `/users/123` `/orders`

HTTP Verbs

Các phương thức HTTP:

- **GET** Lấy dữ liệu
- **POST** Tạo
- **PUT/PATCH** Cập nhật
- **DELETE** Xóa

Phi trạng thái (Stateless)

Mỗi request độc lập, server không lưu trạng thái client giữa các request

Ưu điểm và Nhược điểm của REST

Ưu điểm

Dễ hiểu, dễ tích hợp

Sử dụng các động từ quen thuộc của HTTP

Tận dụng cache HTTP tốt

GET requests có thể được cache, giảm tải server và tăng tốc độ.

Linh hoạt định dạng

Chủ yếu dùng **JSON** (nhẹ, dễ đọc), có thể dùng XML, HTML.

Public APIs phổ biến

Phù hợp cho public APIs, dễ tiếp cận bởi developers bên ngoài.

Nhược điểm

Over-fetching

Lấy **thừa** dữ liệu - endpoint trả về nhiều field hơn cần thiết.

VD: Chỉ cần tên user nhưng API trả cả địa chỉ, phone, email...

Under-fetching

Lấy **thiếu** dữ liệu - phải gọi nhiều requests liên tiếp.

VD: Cần thông tin user + orders + products → 3 requests

N+1 Problem

Khi lấy danh sách + chi tiết item phải dùng nhiều requests.

GraphQL

Bối cảnh

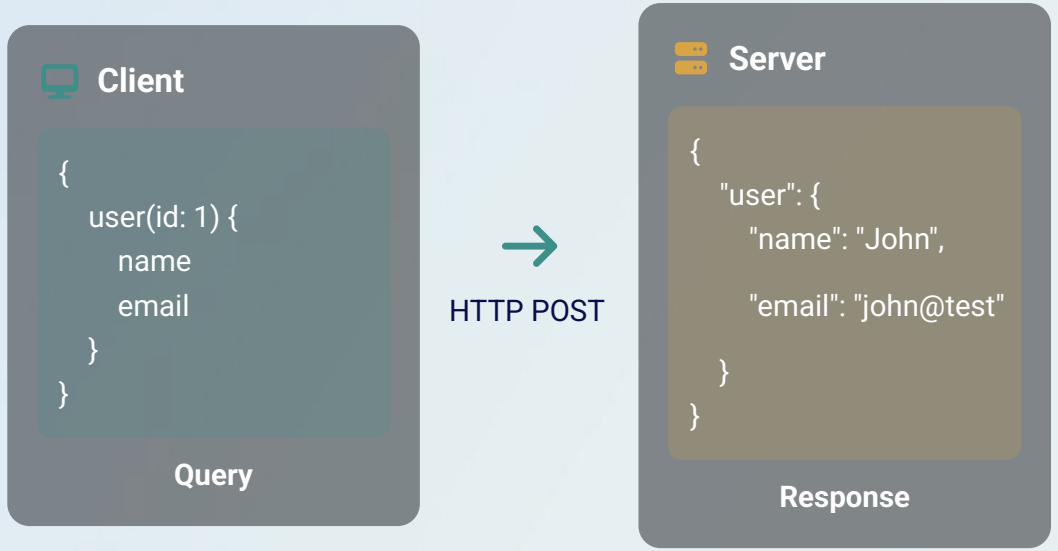
- Facebook tạo ra năm **2012**
- Mã nguồn mở năm **2015**
- Giải quyết mạng yếu trên mobile
- Cấu trúc dữ liệu đồ thị phức tạp

Nguyên lý cốt lõi

Trả **quyền kiểm soát dữ liệu** cho Client:

Client quyết định **những fields nào** cần lấy.

Cách GraphQL hoạt động



REST = Vào nhà hàng gọi **set menu** có sẵn (có thể thừa/thiếu).

GraphQL = Buffet tự chọn - lấy chính xác những gì cần, không thừa một byte.

gRPC

Bối cảnh

- **Google** phát triển
- Liên lạc **hàng nghìn microservices**
- Mở nguồn năm **2015**
- Được **CNCF** đón nhận

Ưu điểm vượt trội

- Tốc độ parse nhanh gấp **nhiều lần** JSON
- Payload nhị phân **siêu nhỏ**
- Latency chỉ **25ms** vs 250ms REST

Công nghệ lõi

HTTP/2

- **Multiplexing:** Có thể thực hiện nhiều requests/ connection
- **Streaming:** 2-way real-time communication
- **Header compression:** Giảm overhead
- **Server push:** Chủ động gửi dữ liệu

Protocol Buffers (Protobuf)

- **Định dạng nhị phân:** Siêu nhỏ, siêu nhanh
- **Schema definition:** .proto files
- **Code generation:** Tự động tạo client/server
- **Khó đọc:** Không đọc được bằng mắt thường

So sánh

Tiêu chí	REST	SOAP	GraphQL	gRPC
Định dạng dữ liệu	**JSON, XML, HTML**	Chỉ XML	JSON	Protobuf (Nhị phân)
Giao thức	HTTP/1.1, HTTP/2	**HTTP, SMTP, TCP**	HTTP/1.1, HTTP/2	Chỉ HTTP/2
Độ khó tích hợp	**Thấp (Dễ nhất)**	Cao	Trung bình	Trung bình - Cao
Latency (thực tế)	~250ms	~300ms	~180ms	**~25ms**
Throughput	20,000 req/s	10,000 req/s	15,000 req/s	**50,000 req/s**
Use Case chính	Public APIs, CRUD	Enterprise, Banking	Mobile, Single Page Apps	Internal Microservices

REST
Public APIs

SOAP
Enterprise

GraphQL
Mobile/SPA

gRPC
Microservices

Công cụ: Postman

Postman

API Development Environment

Công cụ phổ biến nhất để **test, develop, và document APIs**. Hỗ trợ tất cả các loại HTTP requests và nhiều tính năng mạnh mẽ.

Cài đặt

1. Truy cập **postman.com**
2. Download phiên bản phù hợp
3. Cài đặt và đăng ký tài khoản

Demo

- Gọi vào **JSONPlaceholder**
- Fake REST API
- Dùng cho testing và prototyping
- Không cần authentication

Alternative: cURL

- Command-line tool
- Có sẵn trên Linux/Mac
- Windows cần cài thêm

```
curl https://api...
```

Demo 1 - Lấy dữ liệu (GET Request)

Request Details

URL
https://jsonplaceholder.typicode.com/users

Method
GET Đọc dữ liệu

Hành động
Click nút **Send** trong Postman

Phân tích Response

200 **Status Code: 200 OK**
Request thành công

Time
Thời gian phản hồi (ms)

Size
Dung lượng response

response.json


```
[
  {
    "id": 1,
    "name": "Leanne Graham",
    "username": "Bret",
    "email": "Sincere@april.biz",
    "address": {
      "street": "Kulas Light",
      "city": "Gwenborough"
    }
  },
  {
    "id": 2,
    "name": "Ervin Howell",
    ...
  }
]
```

Demo 2 - Gửi dữ liệu (POST Request)

Request Details

URL
https://jsonplaceholder.typicode.com/posts

Method
POST Tạo mới

Headers
Content-Type: application/json

body.json

```
{
    "title": "Hoc API tai Khoa CNTT",
    "body": "Noi dung bai test Postman",
    "userId": 1
}
```

Phân tích Response

201 **Status Code: 201 Created**
Tạo resource thành công

Khác với GET: POST trả về 201 thay vì 200, và response chứa dữ liệu vừa tạo.

HTTP Status Codes

- 2xx Success (Thành công)
- 4xx Client Error (Lỗi client)
- 5xx Server Error (Lỗi server)

Tóm tắt nội dung

API là cầu nối không thể thiếu

Trong phần mềm hiện đại, API đóng vai trò **quan trọng** để các hệ thống, dịch vụ, và ứng dụng giao tiếp với nhau.
Mobile apps, microservices, IoT, hay cloud computing - tất cả đều dựa vào API.

Lựa chọn kiến trúc API phụ thuộc vào **bối cảnh và yêu cầu cụ thể**:

SOAP

Cần chuẩn công nghiệp, bảo mật cao: **Enterprise, Banking, Healthcare**

REST

Hệ sinh thái chung, public, dễ tiếp cận: **Public APIs, CRUD operations**

GraphQL

Tối ưu Frontend/Mobile, giảm over-fetching: **Mobile Apps, Single Page Apps**

gRPC

Tối ưu Backend nội bộ, tốc độ cao: **Internal Microservices**

ANY QUESTIONS?

N+1 problem là gì?

Postman
API Development Environment

Sau khi thực hiện 1 truy vấn ban đầu để lấy N phần tử, hệ thống phải thực hiện thêm N truy vấn phụ để lấy thông tin chi tiết cho từng phần tử.

Biểu hiện ở REST (Under-fetching)

1. Trình duyệt gọi 1 request: `GET /api/posts` → Trả về 10 bài viết (chỉ có `author_id`)
2. Trình duyệt phải gọi thêm 10 requests: `GET /api/users/{id}` để lấy tên tác giả cho 10 bài viết đó.

N+1 trong tầng Database

- Các Framework ORM hiện đại như Hibernate (Java), Entity Framework (C#) hay Mongoose (Node.js) thao tác với Database như thao tác với Object
- Vấn đề nằm ở những câu lệnh SQL thực sự chạy bên dưới. Vòng lặp for tạo ra N kết nối độc lập xuống Database

body.json

```
// 1 query để lấy 100 bài viết
const posts = await Post.find().limit(100);

for (let post of posts) {
    // 100 queries liên tiếp xuống DB!
    const author = await User.findById(post.author_id);
    console.log(post.title, author.name);
}
// Tổng: 101 Queries
```

- Giải pháp ở đây là phải chủ động báo cho ORM biết cần phải Eager Load (tải trước) dữ liệu liên quan bằng một câu JOIN

body.json

```
// Chỉ 1 query (Sử dụng JOIN hoặc WHERE IN dưới DB)
const posts = await
Post.find().limit(100).populate('author_id');

for (let post of posts) {
    console.log(post.title, post.author.name);
}
// Tổng: 1 Query
```

Các kiến trúc API giải quyết N+1 problem

REST API

Mở rộng thiết kế API bằng Query Parameters (Eager Loading từ phía Client)

Ví dụ
Client gọi `GET /api/posts?include=author,comments`

Backend sẽ đọc tham số include để gộp dữ liệu trước khi trả về một cục JSON duy nhất

GraphQL

GraphQL sinh ra là để chống lại N+1, nhưng nếu không cẩn thận, Backend của GraphQL vẫn bị dính N+1 khi truy vấn DB

Sử dụng kỹ thuật Batching với thư viện `DataLoader`.

Cơ chế

DataLoader gom (batch) tất cả các yêu cầu lấy User rải rác trong hệ thống lại, loại bỏ các ID trùng lặp, và chỉ gửi 1 câu lệnh `SELECT * FROM users WHERE id IN (1, 2, 3...)` xuống DB